PipeWire

Introduction

Introduction

- A realtime multimedia data graph
- Works across processes
- Why?
  - Sharing of devices across processes
  - Dynamic routing at runtime
  - Implements format negociation & conversion
  - Modular audio processing, spread across Linux processes
  - Low overhead: shared memory for data and no roundtrip to daemon
- Same abstraction layer (alternatives)
  - `PulseAudio`

Introduction

- `JACK Audio Connection Kit`

- Technical stack: C (`gnu11`), Meson & Ninja

Concepts — objects

- The graph state representation is a list of objects.
- That object list is handled by the `Core` object, hosted by the PipeWire daemon.
- Each connected process is represented by a `Client` object.

Example with `pw-play audio.wav` and

```
pw-record –target pw-play rec.wav
```

:

```
$ pw-cli ls Core
    id 0, type
PipeWire:Interface:Core/4
        object.serial = "0"
        core.name = "pipewire-0"
```

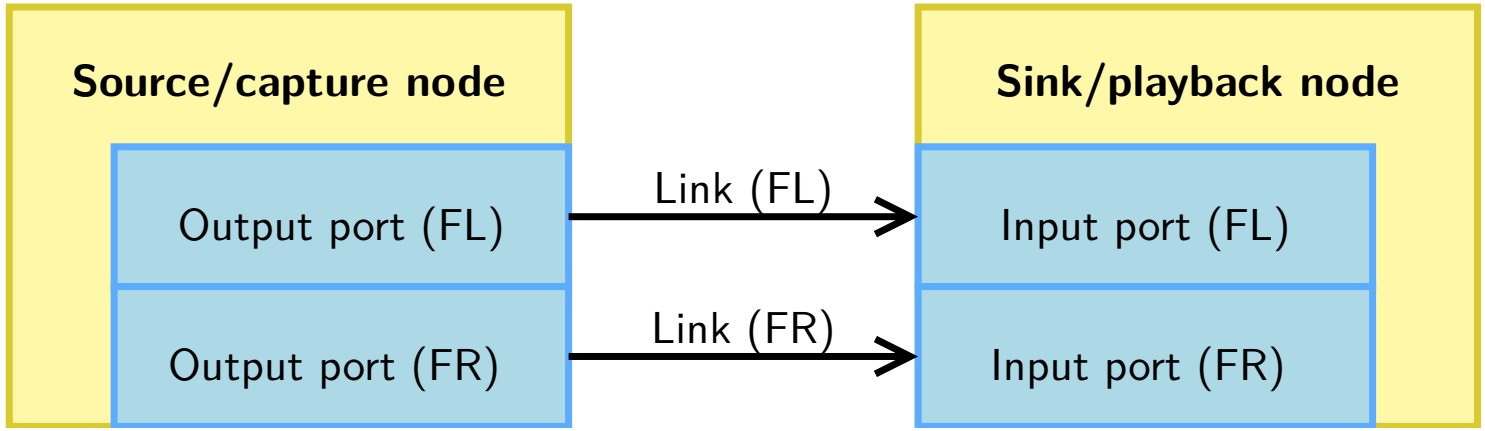
```
$ pw-cli ls Client
    id 35, type
PipeWire:Interface:Client/3
        object.serial = "35"
        pipewire.sec.pid = "2718"
        application.name =
"pipewire"
    id 129, type
PipeWire:Interface:Client/3
        object.serial = "11608"
        pipewire.sec.pid = "466490"
```

Concepts — objects

```
        application.name = "pw-cli"
    id 145, type
PipeWire:Interface:Client/3
        object.serial = "11572"
        pipewire.sec.pid = "465686"
        application.name = "pw-cat"
    id 168, type
PipeWire:Interface:Client/3
        object.serial = "11593"
        pipewire.sec.pid = "466186"
        application.name = "pw-cat"
    ...
```

Concepts — nodes, ports & links (1)

- The graph itself is represented by the following object types:
  - A `Node` processes samples
  - A `Port` represents a node input or output
  - A `Link` connects an output port with an input port



Concepts — nodes, ports & links (2)

```
$ pw-cli ls Node
    id 137, type
PipeWire:Interface:Node/3
        client.id = "145"
        node.name = "pw-play"
        media.class = "Stream/
Output/Audio"
    id 111, type
PipeWire:Interface:Node/3
        client.id = "168"
        node.name = "pw-record"
        media.class = "Stream/
Input/Audio"
    ...

$ pw-cli ls Link
```

```
    id 119, type
PipeWire:Interface:Link/3
        client.id = "33"
        link.output.port = "116"
        link.input.port = "139"
        link.output.node = "137"
        link.input.node = "111"
    id 97, type
PipeWire:Interface:Link/3
        client.id = "33"
        link.output.port = "115"
        link.input.port = "117"
        link.output.node = "137"
        link.input.node = "111"
    ...
```

Concepts — nodes, ports & links (2)

```
$ pw-cli ls Port
    id 116, type
PipeWire:Interface:Port/3
        format.dsp = "32 bit float
mono audio"
        node.id = "137"
        audio.channel = "FL"
        port.alias = "pw-
play:output_FL"
    id 115, type
PipeWire:Interface:Port/3
        format.dsp = "32 bit float
mono audio"
        node.id = "137"
        audio.channel = "FR"
        port.alias = "pw-
play:output_FR"
    id 139, type
PipeWire:Interface:Port/3
        format.dsp = "32 bit float
mono audio"
        node.id = "111"
        audio.channel = "FL"
        port.alias = "pw-
record:input_FL"
    id 117, type
PipeWire:Interface:Port/3
        format.dsp = "32 bit float
mono audio"
        node.id = "111"
        audio.channel = "FR"
        port.alias = "pw-
```

Concepts — nodes, ports & links (2)

```
record:input_FR"
    ...
```

Concepts — object properties and params

- Objects are defined by their ID and type.
- Objects also contain **properties**: a list of string key-value pairs. Those can only be modified by the client hosting the node.
- Some object types also contain **params**. Those might be configurable by other clients.
  - They get used for format negociation & conversion, volume control, etc.

```
$ pw-cli info 94
  type: PipeWire:Interface:Node/3
* properties:
*   application.name = "pw-play"
*   node.name = "pw-play"
*   media.type = "Audio"
*   media.category = "Playback"
*   media.role = "Music"
*   node.rate = "1/44100"
*   node.latency = "4410/44100"
*   node.autoconnect = "true"
*   node.want-driver = "true"
*   media.class = "Stream/Output/
Audio"
*   factory.id = "8"
*   clock.quantum-limit = "8192"
```

Concepts — object properties and params

```
*   library.name = "audioconvert/
libspa-audioconvert"
*   client.id = "151"
*   object.id = "94"
*   object.serial = "2005"
*   ...
* params: (8)
*   3 (Spa:Enum:ParamId:EnumFormat)
r-
*   2 (Spa:Enum:ParamId:Props) rw
*   4 (Spa:Enum:ParamId:Format) rw
*   ...
```

Concepts — devices

- Another object type is `Device` . Those map to physical devices, to which are assigned one or more nodes. Device configuration is done via those objects.
- Providers can be alsa-lib, `BlueZ`, `libcamera`, `V4L`, etc.

Concepts — graph execution logic (1)

- PipeWire structures itself as multiple subgraphs. In each one of those, there is exactly one **driver** node, and zero or more **follower** nodes.
- The driver node is responsible for triggering the start of execution cycles, based on a timer or hardware interrupt for example.
- Each node keeps two counters:
  1. `required` : the number of dependencies on other nodes;
  2. `pending` : how many remaining nodes need to be executed before it can run in this cycle. A value of zero means the node can be executed. Its reset value is `required` .
- Nodes also keep a list of nodes that depend on them (called targets); a node is responsible for decrementing its targets' `pending` counters and signal them using IPC.

Concepts — graph execution logic (1)

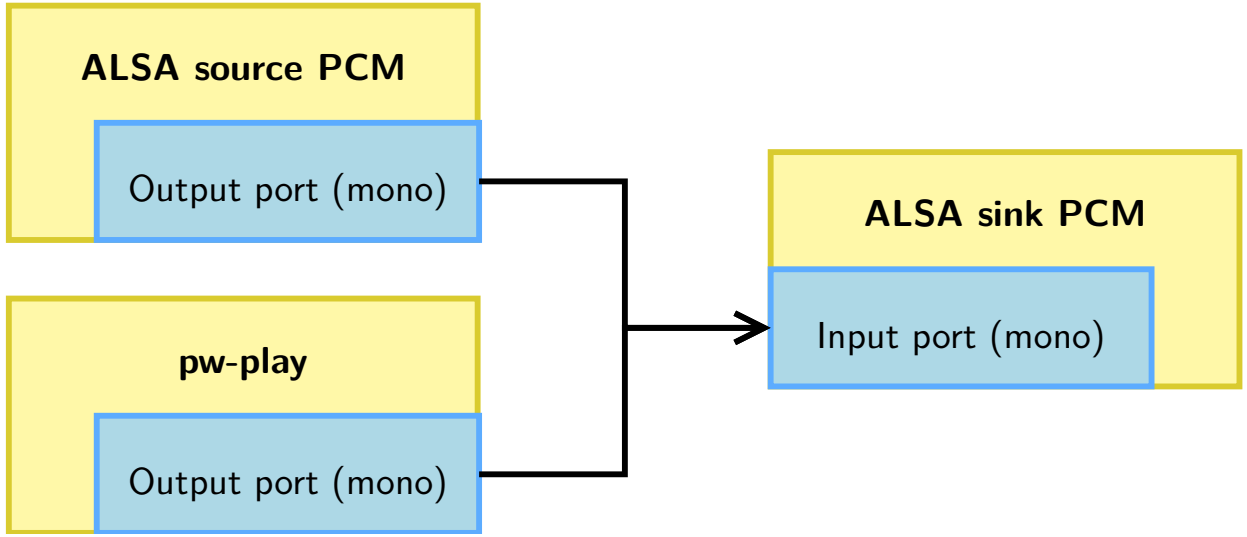
- See the documentation for more details. The graph evaluation is implemented by

```
pw_context_recalc_graph()
```

.

Concepts — graph execution logic (2)

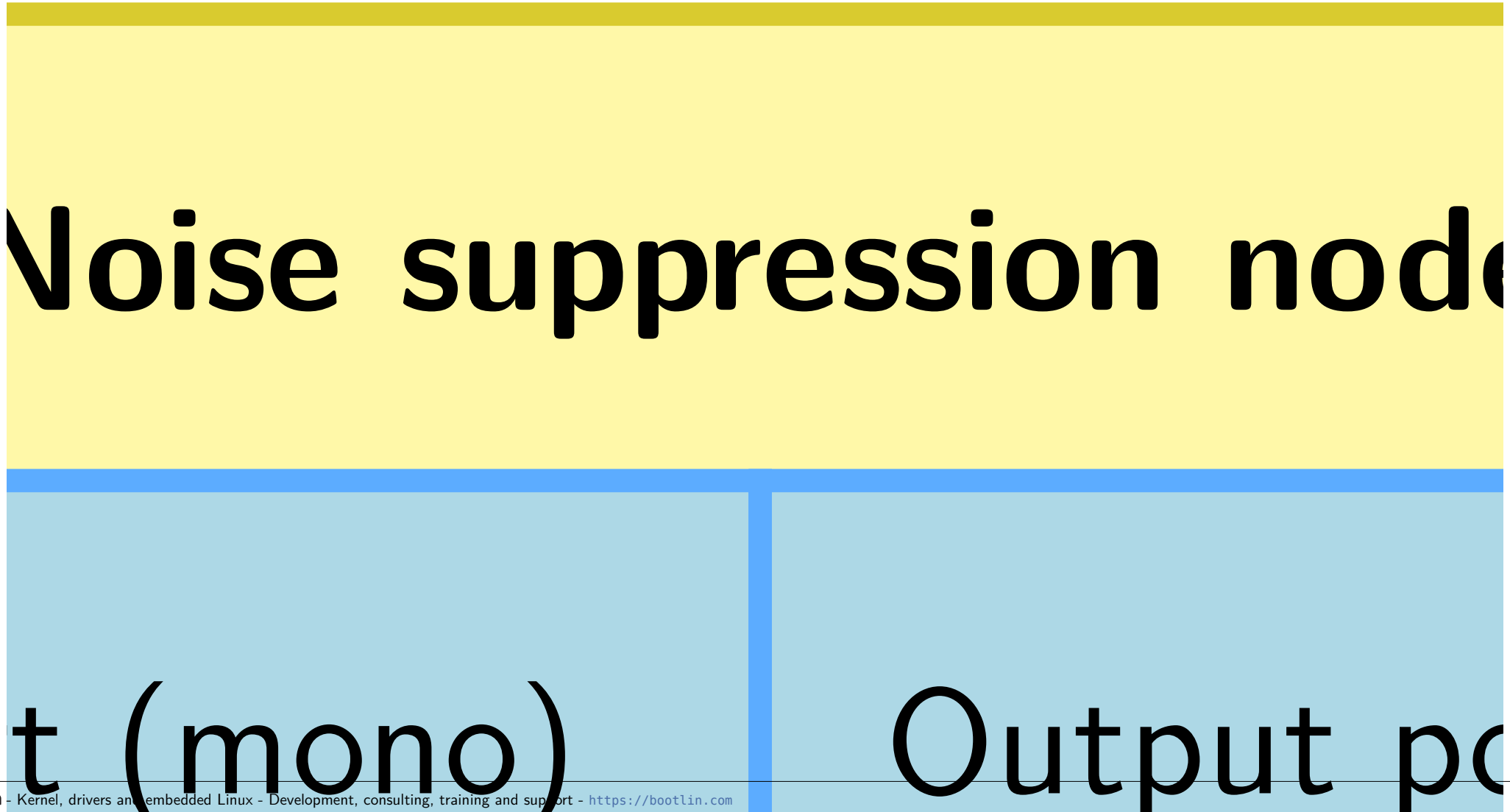
- The driver node is picked based on the `priority.driver` property.
- A good default is to set higher priority to capture driver nodes.



Concepts — graph execution logic (3)

- PipeWire clients and modules can create independent nodes rather than a single one with input and output ports. That allows having multiple subgraphs, each driven by a different driver node.
- **Virtual loopbacks** are such an example: they allow sending samples from a subgraph to another while still decoupling driver clocks.

Concepts — graph execution logic (3)



Concepts — graph execution logic (4)

- The number of samples to be generated during a cycle is called **the quantum**.
- There are global settings for minimum and maximum, and nodes can request specific values for the subgraph they take part in.
- Nodes can also request for a locked quantum: that it does not get changed across recalculations of the graph. This gets used for applications that require fixed quantum (such as the JACK compatibility layer).
- The **rate** is similar: it can be different for each subgraph. The PipeWire config has a list of allowed rates.

Concepts — modules

- Modules are libraries loaded by PipeWire clients to implement various features.
- Example modules:
  - `module-rt` : requests realtime scheduling priority using

    ```
    setpriority(2)
    ```

    and `pthread_setschedparam(3)` .
  - `module-loopback` : create two virtual loopback nodes.
  - `module-protocol-native` : implements the communication between the daemon and clients.
  - `module-profiler` : implements the profiling logic, attached to the daemon.
  - etc.

PipeWire communication protocols — IPC

PipeWire communication protocols — D-Bus optional dependency

- Happens on the session bus
- Flatpak permission support through `XDG Desktop Portal`, see `libpipewire-module-portal`
- Audio device reservation through the `org.freedesktop.ReserveDevice1`, see `libwireplumber-module-reserve-device`
- For Bluetooth support through `BlueZ`, see PipeWire's `libspa-bluez5`

Configuration

{Configuration — location (2)}

- Each client locates and reads its configuration at startup.
- Those configuration files follow a PipeWire-specific format.
- Look-up order:

1. `$XDG_CONFIG_HOME/pipewire/` environment variable, often `~/.config/pipewire/` in distributions item `$sysconfdir/pipewire/` compile-time variable, often `/etc/pipewire/`
2. `$datadir/pipewire/` compile-time variable, often `/usr/share/pipewire/`

end{enumerate}

end{itemize}

{Configuration — location (2)}

begin{itemize}

{Configuration — location (2)}

item A client that loads a config file named `client-rt.conf` will load the first file named as such in the above folders, but will also load all config sections from:

begin{enumerate} item `$datadir/pipewire/client-rt.conf.d/`

3. `$sysconfdir/pipewire/client-rt.conf.d/` item `$XDG_CONFIG_HOME/pipewire/client-rt.conf.d/`

Configuration — sections (1)

- `context.properties` configures the PipeWire instance.
- Most properties target the daemon (`default.clock.allowed-rates`, `default.clock.max-quantum`, etc.) but some also apply to other clients (`log.level`, `mem.mlock-all`, etc.).

```
context.properties = {
    link.max-buffers = 16
    log.level        = 2

    core.daemon = true        #
listening for socket connections
    core.name   = pipewire-0  #
core name and socket name

    # Properties for the DSP
configuration.
```

```
    default.clock.rate          =
48000
    default.clock.allowed-rates =
[ 48000 ]
    default.clock.quantum       =
1024
    default.clock.min-quantum   =
32
    default.clock.max-quantum   =
2048
    default.clock.quantum-limit =
```

Configuration — sections (1)

```
8192
    # ...
}
```

Configuration — sections (2)

- `context.spa-libs` maps plugin features with globs to a SPA library.
- That defines the shared object to be used to implement the given factories. A way to look at this is that keys are interfaces used by PipeWire for various features, and values are the shared objects that implement those.

```
context.spa-libs = {
    # <factory-name regex> =
<library-name>
    # Maps a SPA factory to its
parent library.

    audio.convert.* = audioconvert/
libspa-audioconvert
    avb.*           = avb/libspa-
avb
    api.alsa.*      = alsa/libspa-
alsa
    api.v4l2.*      = v4l2/libspa-
v4l2
    api.libcamera.* = libcamera/
libspa-libcamera
    api.bluez5.*    = bluez5/
libspa-bluez5
    api.vulkan.*    = vulkan/
libspa-vulkan
    api.jack.*      = jack/libspa-
jack
```

Configuration — sections (2)

```
    support.*       = support/
libspa-support
    # ...
}
```

Configuration — sections (3)

- `context.modules` is an array of dictionaries. It lists modules to instantiate, with optional arguments (`args`), `flags` and a conditional expression (`condition`).
- A module can be loaded more than once: it will be instantiated multiple times.
- Two flags exist to turn panics into warnings:
  1. `ifexists` on unknown modules;
  2. `nofail` on module init failures.

```
context.modules = [
    # { name = <module-name>
    #     ( args  = { <key> =
<value> ... } )
    #     ( flags = [ ( ifexists )
( nofail ) ] )
    #     ( condition = [ { <key> =
<value> ... } ... ] )
    # }

    # ...
]
```

Configuration — sections (4)

- `context.modules` example:

```
context.modules = [
    # The profiler module. Allows
application to access profiler
    # and performance data. It
provides an interface that is used
    # by pw-top and pw-profiler.
    { name = libpipewire-module-
profiler }

    # Uses realtime scheduling to
boost the audio thread
    # priorities. This uses RTKit
if the user doesn't have
    # permission to use regular
realtime scheduling.
    { name = libpipewire-module-rt
        args = {
            nice.level    = -11
            #rt.prio      = 88
            #rt.time.soft = -1
            #rt.time.hard = -1
        }
        flags = [ ifexists nofail ]
    }

    # ...
]
```

Configuration — sections (5)

- `context.objects` is an array of dictionaries. It lists objects that should be statically created by this client. This requires a

```
factory
```

  to be used and arguments (`args`) to be passed to it.

- As previously, the `flags` property can configure the reaction to errors. For `context.objects`, only `nofail` exists.
- `condition` also exists for this section.

```
context.objects = [
    # { factory = <factory-name>
    #     ( args  = { <key> =
<value> ... } )
```

```
    #     ( flags =
[ ( nofail ) ] )
    #     ( condition = [ { <key> =
<value> ... } ... ] )
```

Configuration — sections (5)

```
    # }

    # ...
]
```

Configuration — sections (6)

- `context.objects` example:

```
context.objects = [
    # Create a fake source node. It
will be stereo
    # because of its audio.position
property.
    { factory = adapter
        args = {
            factory.name     =
support.null-audio-sink
            node.name        = "my-
mic"
            node.description =
"Microphone"
            media.class      =
"Audio/Source/Virtual"
            audio.position   =
"FL,FR"
        }
    }

    # ...
]
```

Configuration — sections (7)

- `context.exec` is an array of dictionaries. Each entry is an executable that will be run on startup of the client as a child process.
- This used to be the recommended way to run the session & policy manager. This changed and the recommended way is to rely on your init system, be it `systemd` or any other.
- Using this section is therefore **deprecated**, except for simple development environments.

```
context.exec = [
    { path = "/usr/bin/pipewire-
media-session"
        args = ""
        condition = [
            { exec.session-manager
= null }
            { exec.session-manager
= true }
        ] }
]
```

Tools rundown

Tools rundown — the `PIPEWIRE_DEBUG` variable

- Every client listens to the `PIPEWIRE_DEBUG` environment variable which allows overwriting the `log.level` from the configuration file.
- It takes as value the log level:
  - `0` or `X` : No logging is enabled.
  - `1` or `E` : Error logging is enabled.
  - `2` or `W` : Warnings are enabled.
  - `3` or `I` : Informational messages are enabled.
  - `4` or `D` : Debug messages are enabled.
  - `5` or `T` : Trace messages are enabled.
- This should be **the first debugging step** to increase verbosity and therefore better understand why a PipeWire client is facing issues. Careful with `PIPEWIRE_DEBUG=5`

Tools rundown — the PIPEWIRE_DEBUG variable

which most likely will cause underruns issues. Level 3 is often good enough for debugging.

Tools rundown — pw-config

- `pw-config` is a small utility that allows dumping a given config file, taking into account its overrides. It is best used to ensure config changes are effective and overrides are applied as we expect.
- `pw-config paths` lists config paths, including overrides.
- `pw-config list` details all config sections.

```
$ pw-config --name custom.conf paths
{
  "config.path": "/usr/share/pipewire/custom.conf",
  "override.1.0.config.path": "/home/tleb/.config/pipewire/custom.conf.d/
alsa-udev.conf",
  "override.1.1.config.path": "/home/tleb/.config/pipewire/custom.conf.d/
source-rnnoise.conf"
}
```

Tools rundown — pw-dump

- pw-dump prints the graph as a JSON array of all exported objects known to Core .
- Its main goal is to allow sharing the graph's overall state when reporting a bug or describing a situation.
- Filtering: pw-dump takes a parameter which can be an object type (careful, it must be capitalised), ID or name (object.path ,

```
object.serial
```

  or *.name).
- Its output is rather verbose and for more interactive debugging sessions, pw-cli is more adapted.

Tools rundown — `pw-cli` (1)

- `pw-cli` is the main command-line interface tool to interact with PipeWire. It connects to PipeWire as a new client.
- It has two modes: (1) it can either answer to commands given as argument such as `pw-cli help` and stop afterwards or (2) run in interactive mode when given no argument. In that second mode, it also logs new objects that join the core object list.
- `pw-cli help` lists all existing commands. It includes arguments (inbetween square brackets when optional) and command aliases.

Tools rundown — pw-cli (2)

- It can expose many information about the graph:
  - pw-cli ls [<filter>] lists objects with their ID, type and a few of their core properties. <filter> is the same as

    ```
    pw-dump
    ```

    's argument.
  - pw-cli info <filter> gives all possible information about a given object. That includes all of its properties and params.
  - pw-cli enum-params <filter> <param-id> gives the content of a param associated with an object.
- But, it also allows modifying objects:
  - pw-cli set-param <filter> <param-id> <param-json> to edit a param value;

Tools rundown — pw-cli (2)

- pw-cli permissions <client-id> <object> <permission> to modify permissions on a given object.

- As well as creating objects dynamically, that will be hosted by the

```
pw-cli
```

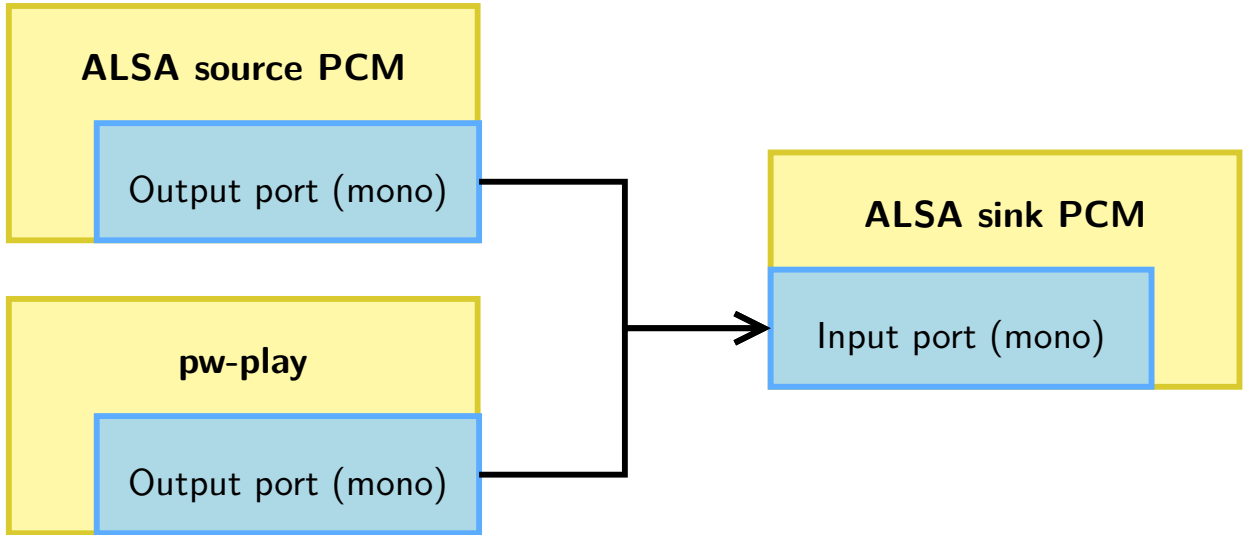
client: load-module , create-device ,

```
create-node
```

, create-link .

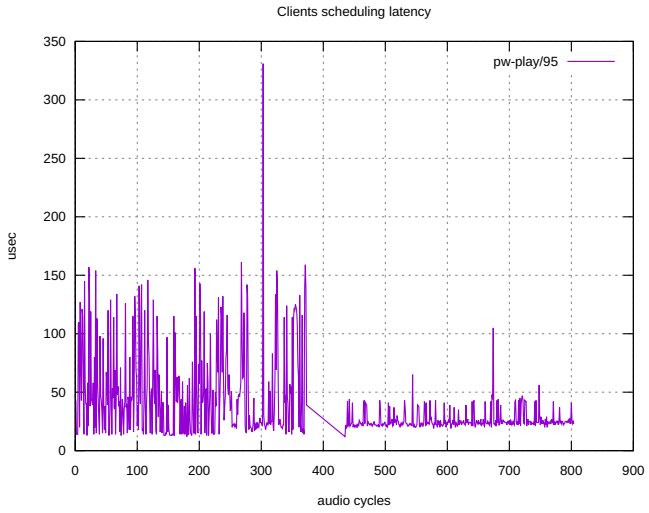
Tools rundown — pw-top

- `top` for PipeWire.
- Appropriate tool to get a quick overview of the current graph nodes and structure.
- Status: `S` for stopped and `R` for running.

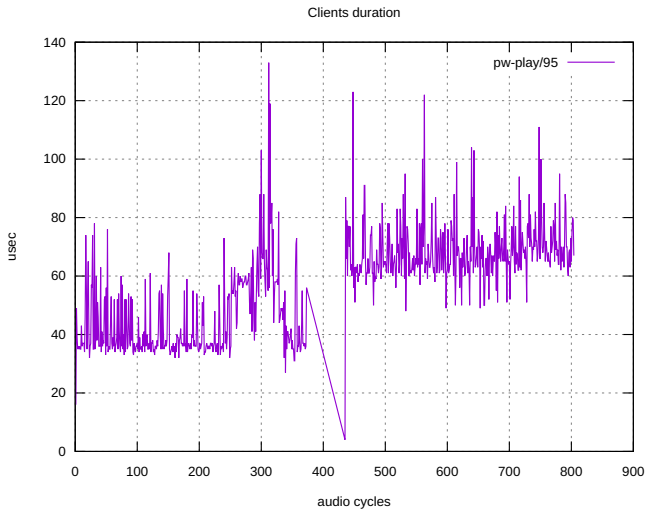


Tools rundown — pw-profiler

- Allows profiling of all running nodes: it records many time durations while running then generates graphs once the command is stopped.
- Here is an example with a single pw-play node, first started with PIPEWIRE_CONFIG_NAME equal to client.conf then with client-rt.conf on a loaded system.



Tools rundown — pw-profiler



Tools rundown — pw-dot

- pw-dot creates a file named pw.dot which is a Graphviz textual graph description file (DOT).
- By default, it connects to the PipeWire daemon and creates a graph representation of the global objects. It can also work from the output of pw-dump using the –json flag.
- That file can be turned into a graphical representation and viewed on a host using: dot -Tsvg pw.dot > pw.svg && xdg-open pw.svg

Tools rundown — pw-cat

- Aliased to pw-play , pw-record and others, it is a simple tool to play or record media files.
- It uses libsndfile for a large audio format support.
- It has many options available to control the exposed props and params:
  - –target allows asking to be routed to a given node;
  - –latency asks for a given latency (therefore buffer size);
  - –quality controls the adaptive resampling;
  - –rate , –channels , –channel-map , –format ,

    –volume

    are self-describing.

Tools rundown — pw-cat

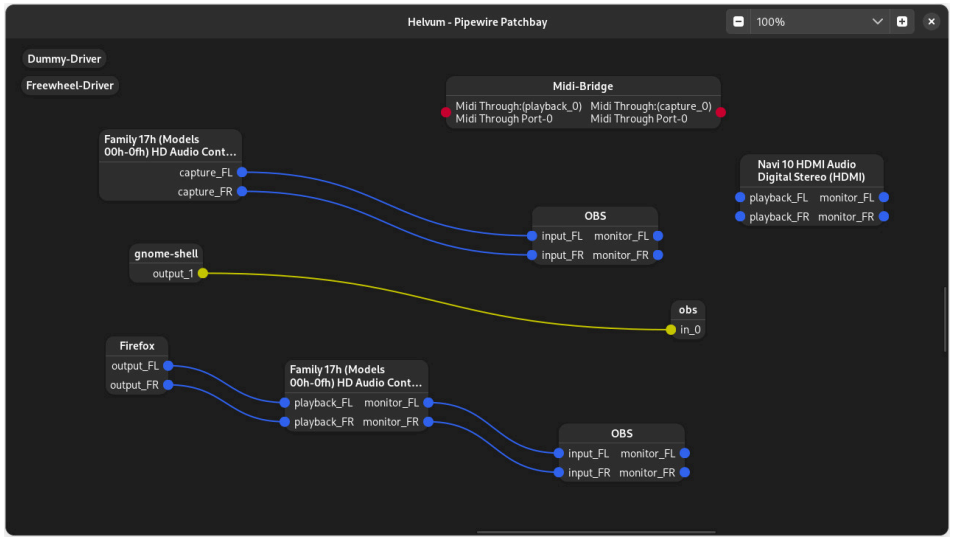
- etc.

Tools rundown — and a few others

- `pw-link` : it allows listing, creating and deleting links.
- `pw-mon` : it monitors and dumps various events: it prints when a global object is added or removed, displays information relative to the `Core` , etc.
- `pw-loopback` : it creates two nodes that act as a virtual loopback.
- `pw-metadata` : it allows editing metadata, which are runtime-writable settings stored by the daemon. The allowed rates and quantum can be controlled at runtime using that method.

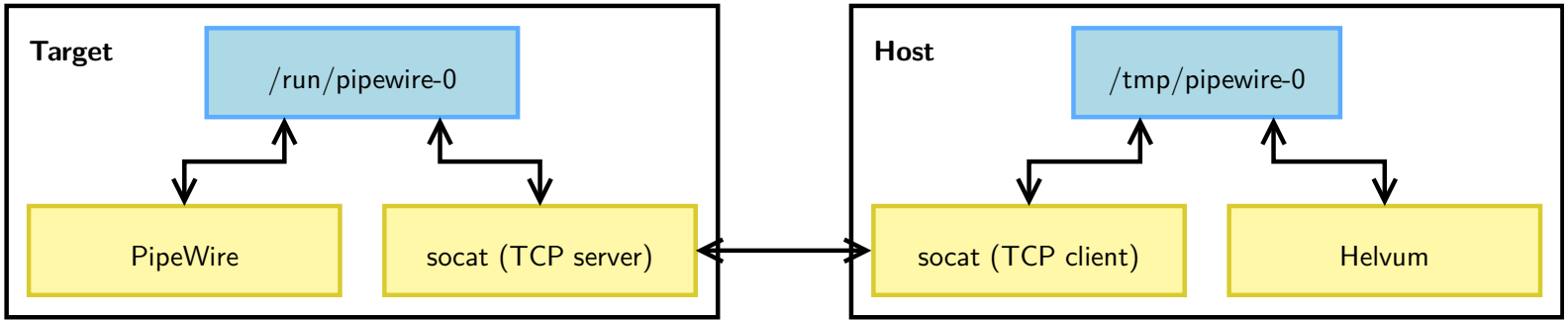
Tools rundown — helvum (1)

- Helvum is a real-time 2D patchbay.
- It gives an overview of the graph with the existing nodes and their ports. It also can create and delete links, allowing manual editing of the graph.



Tools rundown — helvum (2)

- Helvum is a GUI software. We can however run it on our host and monitor our target if we have networking on the target.
- We use socat on the target to bridge the Unix socket from our target daemon over TCP/IP. We then use socat on the host to bridge the TCP/IP to a Unix socket that we will use as our PipeWire Unix socket for Helvum.



Tools rundown — helvum (3)

```
# We run socat on the target,
creating a redirection from the
Unix
# socket /run/pipewire-0 to a TCP/
IP server on port 8000.
ssh $login@$ip "socat TCP4-
LISTEN:8000 UNIX-CONNECT:/run/
pipewire-0" &

# We run socat on the host,
creating the redirection from the
TCP/IP
# port 8000 on the target to the
Unix socket /tmp/pipewire-0 on the
# host.
socat UNIX-LISTEN:/tmp/pipewire-0
```

```
TCP4:$ip:8000 &

# And we connect on the redirected
Unix socket.
PIPEWIRE_RUNTIME_DIR=/tmp helvum
```

Demo 1 — running PipeWire

Demo 1 — introduction

- Demo time!
- We will play audio to an `alsa-lib` device from an audio file.
- We will let our session manager discover ALSA devices and connect an output node to the ALSA sink node.
- The steps will be:

1. Start a PipeWire daemon;
2. Start a WirePlumber daemon;
3. Start a `pw-play` client;
4. Study the graph status using various tools (`pw-dot` ,

```
pw-top
```

Demo 1 — introduction

, `pw-cli` , etc).

Demo 1 — pointers

1. Start a PipeWire daemon.
   - Running `pipewire` without arguments will start a client using `pipewire.conf`, which by default runs in daemon mode.
   - At this state, the graph is rather empty. Objects are mostly modules and factories attached to the core client, and the client objects.
2. Start a WirePlumber daemon.
   - It also picks its config automatically, no arguments required.
   - Once started, we can notice that ALSA devices and attached nodes are created in the graph.
   - Its log level is controlled using `WIREPLUMBER_DEBUG`.
3. Start a `pw-play` client;
   - `pw-play <file>`

Demo 1 — pointers

4. Study the graph status using various tools (`pw-dot` ,

```
pw-top
```

, `pw-cli` , etc).

Demo 2 — PipeWire filter-chains

Demo 2 — introduction

- We will keep our previous setup, but add a client that does equalization on the samples.
- The steps will be:
  1. To create a new configuration file, for the client hosting the effect;
  2. Start a client using this config;
  3. Update links manually to make `pw-play` be routed to the effect, then to the ALSA sink node.

Demo 2 — pointers

1. To create a new configuration file, for the client hosting the effect.
   - Recent PipeWire versions have a `filter-chain.conf` example with snippets for various needs (LADSPA with RNNoise, builtin effects, etc.).
   - When modules spawn objects, they often give their own properties to children, and take arguments to set specific properties for each node. See `capture.props` and `playback.props` .
2. Start a client using this config.
   - `pipewire -c filter-chain.conf`
3. Update links manually to make `pw-play` be routed to the effect, then to the ALSA sink node.
   - This can be done using Helvum with its GUI.

Demo 2 — pointers

- Otherwise, `pw-dot` or `pw-link -links` to get an overview then `pw-link <output-port> <input-port>` to create a new link.

WirePlumber

WirePlumber — session manager concept

- **PipeWire** handles the processing of the media graph.
- An additional layer is required to implement the desired configuration of devices and the connections between nodes. That is implemented by the **session & policy manager**.
- Two known open-source implementations exist:
  - `pipewire-media-session`: the initial implementation, deprecated;
  - `WirePlumber`: recommended implementation.
- **WirePlumber** implements a modular approach: it provides a high-level API and exposes it to `Lua` scripts. Those implement the management logic.
- Technical stack: C, GLib (GObject), Lua engine, Meson & Ninja.
- `Documentation`.

WirePlumber — default behavior

- **WirePlumber** has a default behavior that tries to replicate the PulseAudio behavior, i.e. a desktop setup.
- It enumerates and adds `Device` objects for ALSA, BlueZ and others. It also puts those devices into a best-guess profile.
- Those devices get their associated nodes created automatically.
- Audio routing is based on two default nodes:
  - An `Audio/Sink` node is for applications that want to emit audio. All `Output/Audio` nodes get routed to it.
  - An `Audio/Source` node is for applications that require a microphone input. All `Input/Audio` nodes get routed to it.
- Nodes can also request to be routed to:

1. a target node using `target.object` (for example

WirePlumber — default behavior

```
pw-cat
-target
```

);

2. nothing automatically using `node.autoconnect` . WirePlumber will not create any automatic link, letting any PipeWire client create the desired links;
3. a target node using `target.object` inside `default` metadata (at runtime).

WirePlumber — configuration (1)

- The config lookup logic is the same as PipeWire's.
- Configuration files define components to load when starting. Components include PipeWire modules, WirePlumber modules and **Lua scripts**.
- Lua scripts will **monitor** the PipeWire graph, and **react to events**. Configuration files also defines options passed to scripts, including arrays matching object descriptions to behavior for such objects.
- The default configuration is called `wireplumber.conf` , see

```
/usr/share/wireplumber/wireplumber.conf
```

. Default scripts are located alongside, in `scripts/` .

WirePlumber — configuration (2)

- Let's look at the configuration of the ALSA monitor in

```
scripts/monitors/alsa.lua
```

:

```
-- /usr/share/wireplumber/scripts/monitors/alsa.lua config = {}
config.reserve_device = Core.test_feature ("monitor.alsa.reserve-device")
config.properties = Conf.get_section_as_properties
("monitor.alsa.properties")
config.rules = Conf.get_section_as_json ("monitor.alsa.rules", Json.Array
{})
```

WirePlumber — configuration (3)

- To configure ALSA monitor, using a custom configuration fragments:

```
-- /etc/wireplumber/wireplumber.conf.d/20-alsa-config.conf
monitor.alsa.properties = {
  -- See documentation: man pipewire-devices(7)
  alsa.udev.expose-busy = true
}

monitor.alsa.rules = [
  {
    matches = [ { device.name = "~alsa_card.*" } ]
    actions = {
      update-props = {
        api.alsa.use-acp  = true
        api.acp.auto-port = false
        device.disabled   = false
      }
```

WirePlumber — configuration (3)

```
        }
    }
}
```

WirePlumber — configuration (4)

- Example of disabling D-Bus support:

```
-- /etc/wireplumber/wireplumber.conf.d/10-disable-dbus.conf
wireplumber.profiles = {
   main = {
      support.dbus = disabled

      # Avoid warnings
      support.reserve-device = disabled
      support.portal-permissionstore = disabled
      script.client.access-portal = disabled
      monitor.alsa.reserve-device = disabled
   }
}
```

WirePlumber — permission handling

- Another task of the session & policy manager is **permission management**.
- That is handled, in PipeWire >= 0.3.83, using two PipeWire daemon sockets:
  - Clients joining `pipewire-0-manager` have full permissions, seen using property `pipewire.access = "unrestricted"` .
  - Client joining `pipewire-0` must be given permissions by the session manager, i.e. WirePlumber. Propery `pipewire.access` is `"default"` .
- Permissions can be granted on a per-object-basis for each client. Else each client has a default permission assigned to it.

WirePlumber — DSP filtering on sinks and sources (1)

- WirePlumber now features a way to attach a filter-chain to a sink or source. This allows pre-processing before outputs for example.
- This is called "Automatic Software DSP", see `documentation` about the topic.
- Script implementing this behavior is

```
scripts/node/software-dsp.lua
```

.

WirePlumber — DSP filtering on sinks and sources (2)

- Example configuration:

```
node.software-dsp.rules = [
  {
    matches = [
      { "node.name" = "alsa_output.platform-sound.HiFi__Speaker__sink" }
      { "alsa.id" = "~WeirdHardware*" }
    ]

    actions = {
      create-filter = {
        # Arguments passed to libpipewire-module-filter-chain
        # For inspiration, look into /usr/share/pipewire/filter-chain/
*.conf
        filter-path = "/etc/wireplumber/filter-config.json"
        hide-parent = true
      }
```

WirePlumber — DSP filtering on sinks and sources (2)

```
      }
    }
  ]
```

Demo 3 — interacting with WirePlumber

WirePlumber — demo time! (1)

- We'll use our previous setup, focusing on WirePlumber abilities.
- The steps will be:
  1. Start PipeWire and WirePlumber;
  2. Target a specific node;
  3. Modify the default playback node;
  4. Have a look at device profiles.

WirePlumber — demo time! (2)

1. Start PipeWire and WirePlumber.
   - See demo 1 for explainations.
2. Target a specific node.
   - This is done by nodes using `target.object` (previously

     ```
     node.target
     ```

).
   - It can be a node ID, node name or object path (see WirePlumber scripts for the logic).
   - A node's properties are controlled when spawning it, so by its config or by its client (WirePlumber for example).

WirePlumber — demo time! (2)

3. Modify the default playback node.
   - `wpctl set-default <id>` controls this.
   - Nodes must have `media.class` equal to `Audio/Sink` (or similar) to appear in this list. That does not include filter-chains, which are handled specifically.
4. Have a look at device profiles.
   - Those are params on the device objects. See `EnumProfile` and

```
Profile
```

.

C API

C API — introduction

- `libpipewire` : reference implementation, and currently the only one.
- Allows connecting to the daemon as a client.
- Rust bindings: `pipewire-rs`.
- See `pkg-config` for CFLAGS and LDFLAGS:

```
$ pkg-config --cflags --libs libpipewire-0.3
```

- To initialise the library (logging, randomness, etc.), call:

```
void pw_init(int *argc, char **argv[]);
```

C API — SPA

- A building block is worth mentioning, **Simple Plugin API** (SPA). It contains the following:
  - A `plugin format` encapsulating shared objects, allowing runtime introspection of the plugin content.
  - A Type-Length-Value data container called `POD`. It is header-only, with support for basic types (int, float, string, etc.) and nested types (array, struct, objects).
  - `Utility functions` as header-only: string handling utilities, relaxed JSON parsing (used for config files), a ringbuffer implementation, etc.
  - `Support interfaces` provided by the system, with multiple possible implementations: logging, file-descriptor polling, etc.
- Platform resources (ALSA, bluez5, vulkan, etc.) are exposed as SPA plugins and used internally by PipeWire or WirePlumber.

C API — event-loop

- At the core of each client: an `epoll(2)` -based event-loop is running.
- `pw_main_loop` is a wrapper around `pw_loop` providing a simple-to-use API.

```
/** Create a new main loop. */
struct pw_main_loop *
pw_main_loop_new(const struct spa_dict *props);

/** Get the loop implementation */
struct pw_loop * pw_main_loop_get_loop(struct pw_main_loop *loop);

/** Destroy a loop */
void pw_main_loop_destroy(struct pw_main_loop *loop);

/** Run a main loop. This blocks until ref pw_main_loop_quit is called */
int pw_main_loop_run(struct pw_main_loop *loop);
```

C API — event-loop

```c
/** Quit a main loop */
int pw_main_loop_quit(struct pw_main_loop *loop);
```

C API — context

- A `pw_context` instance is at the heart of the C API. It allows connection to the daemon and it manages locally available resources.
- It does the following:
  - Parsing of the appropriate configuration.
  - Start of the processing thread & associated data loop.
  - Handling of local resources: memory pool, work queue, **proxies**, local modules

```
/** Make a new context object for a given main_loop */
struct pw_context * pw_context_new(struct pw_loop *main_loop,
                struct pw_properties *props, size_t user_data_size);

/** Connect to a PipeWire instance */
```

C API — context

```
struct pw_core * pw_context_connect(struct pw_context *context,
                struct pw_properties *properties, size_t user_data_size);
```

C API — proxies

- Think as **proxies** as file descriptors for PipeWire objects. They are local references to global PipeWire objects.
- The equivalent on the daemon side are called **resources**.
- A client starts with two proxies:
  1. One pointing to the `Core` object.
  2. Another one to the global `Client` object that represents itself.

C API — registry

- The PipeWire daemon handles a list of objects. Those are known as **global** objects and are represented by `pw_global` structures.
- `pw_registry` is a singleton structure that allows clients to track existing globals. It works by registering a callback to be called on new global object events.

```
struct pw_registry_events {
#define PW_VERSION_REGISTRY_EVENTS  0
        uint32_t version;
        void (*global) (void *data, uint32_t id, uint32_t permissions,
                const char *type, uint32_t version,
                const struct spa_dict *props);
        void (*global_remove) (void *data, uint32_t id);
};

struct pw_registry * pw_core_get_registry(struct pw_core *core,
        uint32_t version, size_t user_data_size);
```

C API — registry

```
void pw_registry_add_listener(struct pw_registry *registry,
        struct spa_hook *hook, struct pw_registry_events *events,
        void *data);
```

C API — example 1, monitoring global objects

```
/* We will run indefinitely, getting events for each added and removed
global
 * object.
 *
 * An influx of Registry::Global events will come in at the start to list
all
 * already-existing globals. Use the Core::Sync method and Core::Done event
to
 * know when that initial sync is done. See pw_core_sync(). */
#include <pipewire/pipewire.h>

static void registry_event_global(void *data, uint32_t id, uint32_t
permissions,
                const char *type, uint32_t version, const struct spa_dict
*props) {
        printf("object added: id:%uttype:%s/%dn", id, type, version);
```

C API — example 1, monitoring global objects

```
}

static void registry_event_global_remove(void *data, uint32_t id) {
        printf("object removed: id:%un", id);
}

static const struct pw_registry_events registry_events = {
        PW_VERSION_REGISTRY_EVENTS,
        .global = registry_event_global,
        .global_remove = registry_event_global_remove,
};
```

C API — example 1, monitoring global objects

```
int main(int argc, char **argv) {
  pw_init(&argc, &argv);

  struct pw_main_loop *loop = pw_main_loop_new(NULL);
  struct pw_context *context = pw_context_new(pw_main_loop_get_loop(loop),
NULL, 0);
  struct pw_core *core = pw_context_connect(context, NULL, 0);
  struct pw_registry *registry = pw_core_get_registry(core,
PW_VERSION_REGISTRY, 0);

  struct spa_hook registry_listener;
  spa_zero(registry_listener);
  pw_registry_add_listener(registry, &registry_listener, &registry_events,
NULL);

  pw_main_loop_run(loop);
```

C API — example 1, monitoring global objects

```
  pw_proxy_destroy((struct pw_proxy*)registry);
  pw_core_disconnect(core);
  pw_context_destroy(context);
  pw_main_loop_destroy(loop);

  return 0;
}
```

C API — node implementations

- Implementing a raw node is not straight-forward, requiring to implement many book-keeping methods (see

```
struct spa_node_methods
```

).

- PipeWire provides two abstractions for implementing nodes:
  - `pw_filter` : DSP-type work, works on raw `f32` samples, without additional buffering.
  - `pw_stream` : more high level, it provides the following features:
    - **Buffering:** a stream can emit more samples than the current cycle quantum and those will be buffered.
    - **Format negociation:** the client can expose multiple supported formats and negociation will occur when changing from idle to running.
    - **Format conversion:** sample type, planar/interleaved, channel mapping, rate resampling.

C API — node implementations

- See example implementations of source nodes:
  - Filter: `src/examples/audio-dsp-src.c`
  - Stream: `src/examples/audio-src.c`

C API — pw_filter process event

```c
static void on_process(void
*userdata, struct spa_io_position
*position) {
  struct data *data = userdata;
  double *acc = data->out_port-
>accumulator;
  uint64_t n_samples = position-
>clock.duration;

  /* Fetch the sample buffer. The
first argument is the port user
data
   * (as returned by
pw_filter_add_port), it is used to
identify our
   * port (think container_of). */
```

```c
  float *out =
pw_filter_get_dsp_buffer(data-
>out_port, n_samples);
  if (out == NULL)
    return;

  for (uint64_t i = 0; i <
n_samples; i++) {
    *acc += 2 * M_PI * 440 /
44100;   /* Grow our accumulator */
    *acc = remainder(*acc, 2 *
M_PI); /* Avoid overflows */
    *out++ = sin(*acc) *
0.7;         /* Compute a sample */
  }
}
```

C API — pw_filter process event

```
static const struct
pw_filter_events filter_events = {
  PW_VERSION_FILTER_EVENTS,
  .process = on_process,
};
```

C API — pw_stream process event

```
static void on_process(void
*userdata) {
  struct data *data = userdata;

  struct pw_buffer *b =
pw_stream_dequeue_buffer(
      data->stream);
  assert(b != NULL);

  struct spa_buffer *buf = b-
>buffer;
  uint8_t *p = buf->datas[0].data;
  assert(p != NULL);

  int stride = sizeof(float) *
CHANNELS;
```

```
  int n_frames = SPA_MIN(b-
>requested,
      buf->datas[0].maxsize /
stride);

  fill_f32(&data->accumulator, p,
n_frames);

  buf->datas[0].chunk->offset = 0;
  buf->datas[0].chunk->stride =
stride;
  buf->datas[0].chunk->size =
n_frames * stride;

  pw_stream_queue_buffer(data-
```

C API — pw_stream process event

```
>stream, b);
}
```

```
#define CHANNELS 2
#define FREQ     440
#define RATE     44100

static void fill_f32(float *acc,
float *dest,
    int n_frames) {
  for (int i = 0; i < n_frames; i+
+) {
    *acc += M_PI_M2 * FREQ / RATE;
    *acc = remainder(*acc, 2 *
M_PI);

    float val = sin(*acc) * 0.7;
    for (int c = 0; c < CHANNELS;
c++)
```

C API — pw_stream process event

```
        *dst++ = val;
    }
}

static const struct
pw_stream_events stream_events = {
    PW_VERSION_STREAM_EVENTS,
    .process = on_process,
};
```

Going further

Going further

- For MIDI support, see `pw-cat –midi` , `pw-mididump` and the documentation.
- For the PulseAudio compatibility layer, see `module-protocol-pulse` and this documentation page.
- For the JACK compatibility layer, look at `pw-jack` .
- For video support, see many examples in `src/examples/` .
- For audio over IP, see modules `roc-*` , `pulse-tunnel` ,

```
netjack2-*
```

, `rtp-*` , `protocol-simple` , `avb` .

- To understand why timer-based audio scheduling (`tsched`) is useful, see this blog post.